

3966R0

2026-01 Library Evolution Poll Outcomes

Published Proposal, 2026-02-22

Authors:

[Inbal Levi - Library Evolution Chair](#) (Microsoft LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Andreas Weis - Library Evolution Assistant Chair](#) (ekxide IO GmbH)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

- 3.1 Poll 1: Send "[P3505R2] Fix the default floating-point representation in `std::format`" to Library Working Group for C++29 (with a recommendation for a DR).
- 3.2 Poll 2: Apply the resolution in "[P3826R3] Fix or Remove Sender Algorithm Customization" to resolve the NB comments: US 207-328, FI-331, CA-358, FR-031-319, US 202-326, and send to Library Working Group for C++26.
- 3.3 Poll 3: Apply the resolution in "[P3450R0] Extend `std::is_within_lifetime`" to resolve the NB comment: US 82-145, and send to Library Working Group for C++26.

References

Informative References

§ 1. Introduction

In 2026-01, the C++ Library Evolution group conducted a series of electronic decision polls [\[P3965R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 19 people participated in the polls. Thank you to everyone who participated, and to the papers' authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send "[P3505R2] Fix the default floating-point representation in <code>std::format</code> " to Library Working Group for C++29 (with a recommendation for a DR).	11	4	0	0	0	Strong consensus in favor
Poll 2: Apply the resolution in "[P3826R3] Fix or Remove Sender Algorithm Customization" to resolve the NB comments: US 207-328, FI-331, CA-358, FR-031-319, US 202-326, and send to Library Working Group for C++26.	4	7	2	1	1	Consensus in favor
Poll 3: Apply the resolution in "[P3450R0] Extend <code>std::is_within_lifetime</code> " to resolve the NB comment: US 82-145, and send to Library Working Group for C++26.	7	6	2	1	0	Consensus in favor

All the polls have consensus in favor and the papers will be forwarded to LWG.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[P3505R2] Fix the default floating-point representation in `std::format`" to Library Working Group for C++29 (with a recommendation for a DR).

While this is a breaking change, it's the right thing to do to improve readability and make output more consistent with other languages. We should make this a DR.

— Strongly Favor

I am in favor P3505R3, which actually shows the option that was selected. And can we please fix the paper title, since we fix `to_chars` (and `std::format` is just fall-out)?

— Weakly Favor

§ 3.2. Poll 2: Apply the resolution in "[P3826R3] Fix or Remove Sender Algorithm Customization" to resolve the NB comments: US 207-328, FI-331, CA-358, FR-031-319, US 202-326, and send to Library Working Group for C++26.

Algorithm customization is important and should be fixed. I'm confident Eric has figured out the problem.

— Strongly Favor

Seems like we have to do this. It gives me no particular pleasure to lose the early-diagnostics possibility for the error/stop channels, but we just have to look at providing such a facility separately for those who choose to use it.

— Weakly Favor

This is superior to previous shape of CPs, and I believe this is the right direction.

— Weakly Favor

Would rather "remove" anything in this area than "fix" it; but that's not on the table.

— Neutral

Customization is being designed very late with little or no implementation experience. It would be better to postpone this to C++29.

— Weakly Against

Too late for such a significant change.

— Strongly Against

§ 3.3. Poll 3: Apply the resolution in "[P3450R0] Extend `std::is_within_lifetime`" to resolve the NB comment: US 82-145, and send to Library Working Group for C++26.

A reasonable extension that will help `constexpr` type erasure

— Strongly Favor

While I'd prefer an overload that takes a reference, there is nothing that prevents this from being added later.

— Strongly Favor

Participating in the LEWG review helped me understand why we need this.

— Strongly Favor

A useful and straightforward extension of `std::is_within_lifetime`.

— Weakly Favor

Not a pressing issue.

— Neutral

The design is dubious as the default parameter is "void" which does not seem to make semantic sense in the context of lifetimes. No objections to the overall intend, though.

— Weakly Against

§ References

§ Informative References

[P3965R0]

Inbal Levi; et al. *2026-01 Library Evolution Polls*. 16 January 2026. URL:
<https://wg21.link/P3965r0>